# Module - 1
## Data Structures

Nithin Kumar
Asst. professor
Dept. of CSE
VVCE, Mysuru

A "Data-Structure" is a particular way of organizing data in a Computer so that it can be used Effectively.

Ex:- We can Store a list of Items having Same data-type using the array data Structure.

* Types of Data Structures :- There are two classes of data Structures Such as
  * primitive Data Structures
  * Non-primitive Data Structures

* primitive Data Structures → are the Basic data Structures that directly operates upon the Machine Instructions.
→ primitive Data Structures are fundamental datatypes which are Supported by a programming language.
→ "Integers", "floating-point numbers", "character Constants", "String" & "pointers" Come under this category.

* Non-primitive Data Structures → are More Complicated data-Structures & derived from primitive data Structures.
→ They Emphasize on Grouping Same or different data Items with relationship between Each data Item.
→ Non primitive data Structures are also called "Compound-data Structures".
→ "Array", "Lists", & "files" Come under this category.

Nithin, VVCE Mysuru

Scanned with CamScanner

→ Non-primitive data Structures are classified as,

* Linear Data Structures
* Non-Linear Data Structures

* Linear Data Structures → Data Structure where its Values or Elements are Stored in Sequential or Linear order is called "Linear Data Structures".
Ex:- Arrays, Stacks, Queue, Linked Lists.

* Non-Linear Data Structures → Data Structures where its Values or Elements are not Stored in Sequential or Linear order is called "Non-Linear Data Structures".
Ex:- Trees & Graphs.

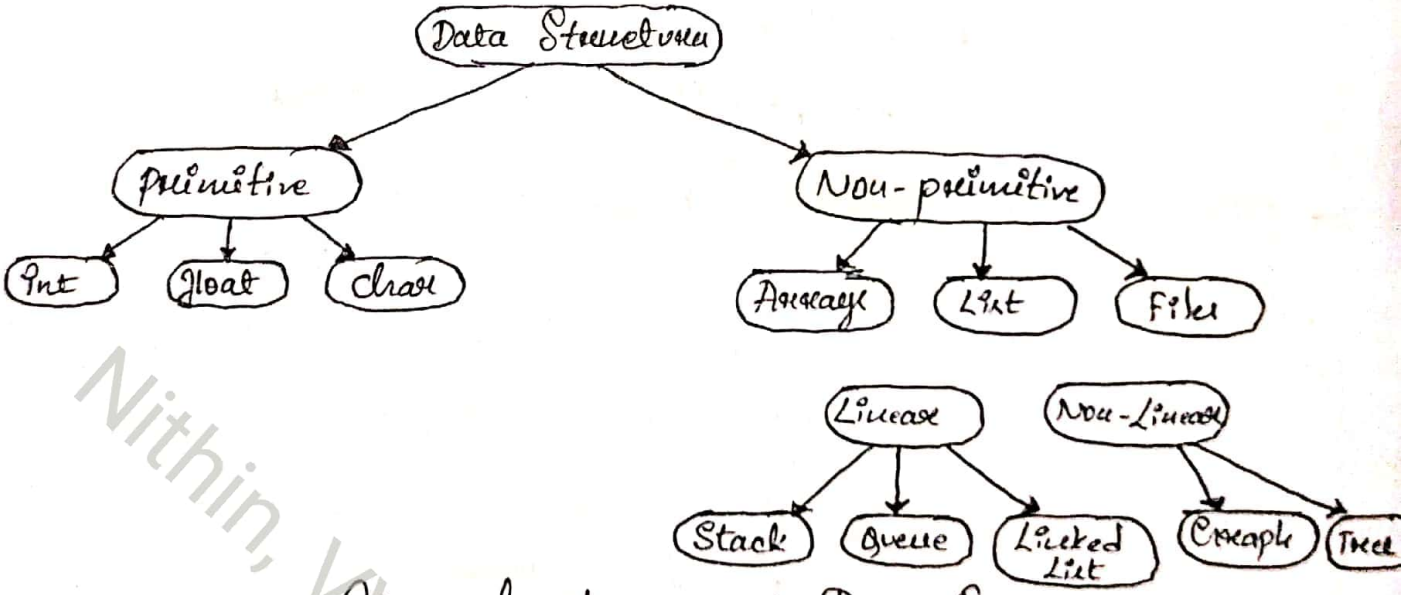



Fig:- Classification of Data Structures

* Data Structure Operations :- The Basic operations that can be performed on Data Structures are as follows

* Insertion → addition of a new Element into a data Structure.

* Deletion → Removal of a data Element from a data Structure if it is found.

* Searching → Searching for the Specified data Element in a data Structure.

Nithin, VCE Mysuru

Scanned with CamScanner

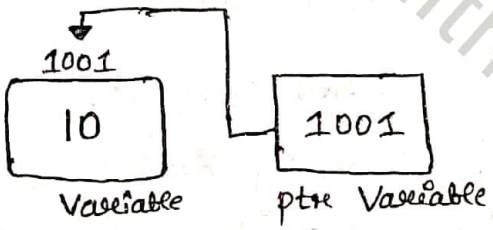
* Traversal → Traversal of a data structure means processing all the data Elements present in it.

* Sorting → Arranging data Elements of a data structure in Specified order is called Sorting.

* Merging → Combining Elements of two Similar data structure to form a new data structure of same type.

* pointers :- The "pointer" is a Variable which Stores the Address of another Variable.

→ pointers can be used to access & Manipulate data Stored in the Memory.




The Concept of Storing Memory address in a Variable & Accessing the value available at that address is known as a "pointer Variable".

* Declaring & Initializing the pointer Variable → pointer Variable Must be declared using the same datatype present in the address that it Stores.
The process of Assigning the address of a Variable to pointer Variable is known as Initialization.

Syntax :- Datatype *ptr_Variable;
ptr_Variable = & Var_name;

Example :- int x, *ptr;
ptr = &x;

* Dereferencing → Dereferencing a pointer means getting the value that is stored in the Memory location pointed by

Nithin, VVCE, Mysuru

Scanned with CamScanner

pointers. The operator '*' is used to do this & is called as "Dereferencing operator".

Ex6
```
int x = 10;
int *ptr;
ptr = &x;
y = *ptr;
```

In the above Example, the value '10' is Assigned to 'y' from 'x' using pointers.

* pointers in Arrays → if we declare pointer, then we can make the pointer to point to the Array by the following Assignment.

```
ptr_Variable = Arr_name;
        or
ptr_Variable = &Arr_name[0];
```

We can Access Every value of Array by Incrementing the pointer Variable to Move from one Element to another.

* pointers as Function Arguments → The process of Calling a function using pointers to pass the address of Variable is known as "Call by Reference".

Example:-
```
#include <stdio.h>
void Simple (float *, float *, float *);
void main ()
{
    float p, t, r, Si, *p1, *p2, *p3;
    printf ("Enter ptr value");
    scanf ("%f %f %f", &p, &t, &r);
    p1 = &p; p2 = &t; p3 = &r;
    Simple (p1, p2, p3);
}
void Simple (float *p1, float *p2, float *p3)
{ float Si;
  Si = (*p1 * *p2 * *p3)/100;
```

Nithin, VVCE

Scanned with CamScanner

```
printf("Simple Interest = %f", SI);
}
```

* **Dynamic Memory Allocation :-** In your program you may wish to acquire space in which you will store information. When you write your program you may not know how much space you will need, how do you with to allocate some very large area that may never be required.
To Solve this problem by using "dynamic memory allocation", we can allocate storage at run-time using pointer.

→ Dynamic Memory allocation can be defined as a procedure in which the size of a data structure will be changed during the runtime.

→ C provides some functions to achieve these task, there are 4 Library functions provided by C defined under <Stdlib.h> header file to facilitate memory allocation in C programming:

* Malloc()
* Calloc()
* Realloc()
* Free()

* **Malloc()** → "Malloc" or "Memory allocation" method in C used to dynamically allocate a single large block of Memory with the specified size.

Syntax : ptr_variable = (datatype *) malloc (n * sizeof (datatype));

Example : ptr = (int *) malloc (n * sizeof (int));

* **Calloc()** → "Calloc" or "Contigous allocation" method in C used to dynamically allocate the specified number of blocks of memory of the specified type.

Syntax : ptr_variable = (datatype *) calloc (n, sizeof (datatype));

Nithin, VVCE Mysuru

Scanned with CamScanner

Example : ptr = (float *) calloc (n, sizeof (float));

* Realloc() → "Realloc" or "Re-allocation" method in C is used to dynamically change the Memory allocation of a previously allocated Memory.

In other words, if the Memory previously allocated with the help of "Malloc" or "Calloc" is insufficient, Realloc can be used to dynamically re-allocate Memory.

Syntax : ptr_Variable = (datatype *) Realloc (ptr_Variable, Size);

Example : ptr = (int *) Realloc (ptr, n);

* Free() → "Free" method in C is used to dynamically de-allocate the Memory. The Memory allocated using function malloc() & calloc() will not de-allocate on their own.

Hence the free() Method is used, whenever the dynamic Memory allocation take place. It helps to reduce wastage of Memory by freeing it.

Syntax : free (ptr_Variable);

Example : free (ptr);

During the Memory allocation, when the requested memory is available, a pointer to the start of an area of Memory of the required size is returned. If the Memory is not available, the pointer NULL is returned.

* Example program → to demonstrate the allocation, reallocation & de-allocation in C.

```
#include <stdio.h>
#include <stdlib.h>
Void main()
{
int *arr, i, n, m;
printf("Enter the no of Elements");
```

Nithin, VVCE

Scanned with CamScanner

```
scanf ("%d", &n);
arr = (int *) malloc (n * sizeof(int)); (or) arr = (int *) calloc (n, sizeof(int));
if (arr == NULL)
{ printf ("Memory allocation failed");
}
Else
{ printf ("Enter the Array Elements");
  for (i=0; i<n; i++)
    { scanf ("%d", &arr[i]);
    }
  printf ("Enter Additional M Elements");
  scanf ("%d", &m);
  arr = (int *) realloc (arr, m);
  printf ("Enter the Additional array Elements");
  for (i=n; i<n+m; i++)
    { scanf ("%d", &arr[i]);
    }
}
printf ("The Array Elements are");
for (i=0; i<n+m; i++)
  { printf ("%d", arr[i]);
  }
free (arr);
}
```

* Common Errors of Dynamic Memory allocation

* Not checking for allocation failures → Memory allocation is not Guaranteed to Succeed & May instead return a NULL pointer. Using the returned Value without checking, will invoke undefined behaviour leads to Crash.
* Memory Leaks → Failures to deallocate Memory using "free" leads to buildup non-reusable Memory, which is no longer used by program. This wastes Memory resources & can lead to allocation failures.

Nithin, VVCE Mysuru

Scanned with CamScanner

* Logical Errors → All allocations Must follow the Same pattern.

Malloc() or Calloc() → Realloc() → Free()

Failure to adhere this pattern one Calling "free" twice Etc. Causes a "Segmentation Fault" & results in Crash of the program.

* Data Abstraction :- Data abstraction refers to providing only Essential Information to the outside world & hiding their back-ground details, i.e to represent the needed Information in pro-gram without presenting the details.

→ Lets take one real life Example of a TV, which you can turn on & off, change the Channel, adjust the Volume, but you do not know its Internal details, that is, you do not know how it receives Signals over the air or how it translates them & finally display them on the Screen.

* Abstract Data types → ADT is a type for objects whose beha-viour is defined by a "Set of Value" & a "Set of operations".

→ The definition of ADT only Mentions what operations are to be performed but not how those operation will be Imple-mented.
It does not Specify how data will be organized in Memory & what algorithms will be used for Implementing the opera-tions.

→ It is called "abstract" because it gives an Implementation Inde-pendent views.

* ADT of Natural Number → an ordered Subrange of the Integers Starting at zero & Ending at the Maximum Inte-ger (INT_MAX) on the Computer.

* Functions on Natural Numbers :-

Nithin VC Mysuru

Scanned with CamScanner

* Natural Number zero ( ) ⇒ 0
* Boolean Is Zero (x) ⇒ If (x) return FALSE Else return TRUE
* Boolean Equal (x, y) ⇒ If (x == y) return TRUE
  Else return FALSE
* Natural Number Succesor (x) ⇒ If (x == INT_MAX) return x
  Else return x+1
* Natural Number Add (x, y) ⇒ If ((x+y) <= INT_MAX) return x+y
  Else return INT_MAX
* Natural Number Subtract (x, y) ⇒ If (x < y) return 0
  Else return x-y

* ADT of polynomial → A polynomial object is a Homogenous ordered List of pairs <Exponent, Co-efficient> where Each Co-efficient is unique.
* Functions on polynomial :-
* polynomial zero ( ) ⇒ return the polynomial p(x) = 0
* Boolean Is zero (poly) ⇒ if (poly == 0) return TRUE
  Else return FALSE
* Coefficient Coeff (poly, Exp) ⇒ if (Exp ∈ poly) return Co-eff
  Else return FALSE
* Exponent Lead Exp (poly) ⇒ return the highest degree present in poly.
* polynomial Remove (poly, Exp) ⇒ if (Exp ∈ poly) remove the Corresponding term Else return FALSE.
* polynomial Add (poly1, poly2) ⇒ return poly1 + poly2;
* polynomial Multiply (poly1, poly2) ⇒ return poly1 * poly2;

Within VCE Mysuru

Scanned with CamScanner

* ADT of Array → A Set of pairs <index, value> where for Each value of index there is a value. index is a finite ordered Set of one or More dimensions.

* Functions on Array :-

* Array Create (j, list) ⇒ return an Array of j dimensions Items are undefined.

* Item Retrieve (A, i) ⇒ return the Item Associated with index value i in Array A
Else return Error.

* Array Store (A, i, x) ⇒ return an Array that is identical to array A Except the new pair <i, x) has been inserted
Else return Error.

* ADT of Sparse Matrix → A set of triples <row, column, value>, where row & column are Integers & form a unique Combination.

* Functions on Sparse Matrix :-

* SparseMatrix Create (MaxRow, MaxCol) ⇒ return a SparseMatrix that can hold upto MaxItem = MaxRow × MaxCol

* SparseMatrix Transpose (a) ⇒ return the Matrix produced by Interchanging row and column.

* SparseMatrix Add (a, b) ⇒ If the dimensions of a & b are the Same, return the Matrix produced.
Else return Error.

* SparseMatrix Multiply (a, b) ⇒ If the no of Columns in a Equals no of rows in b, return the Matrix produced.
Else return Error.

Nithin VTU CE Mysuru

Scanned with CamScanner

* Dynamically allocated Arrays :- Since C is a Structured language, It has some fixed rules for programming. One of them Such -des changing the Size of an array.

→ Consider the following

int arr[9];

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| arr = | 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 |

As it can be seen that the length (Size) of the Array above Made is 9. But what if there is a requirement to change this length?

This Situation can be addressed by using dynamic Memory allocation functions (Malloc(), calloc(), realloc()) for Memory allocation Such arrays are called "dynamically allocated Arr -ay".

"Dynamically allocated Arrays" in which the Size of a Array is changed during the Runtime.

* 1-D Dynamically Allocated Array → In which the Size of the Array will be decided during the Runtime based on the User Input (n). Malloc() or Calloc() will be used to allocate the required Memory, which can be Modified by using realloc().

* Example - program: to read & print 1-D Array dynami -cally using Malloc() or Calloc().

```
#include <stdio.h>
#include <stdlib.h>
Void main()
{
int *arr, i, n;
printf("Enter the no of Elements");
Scanf("%d", &n);
```

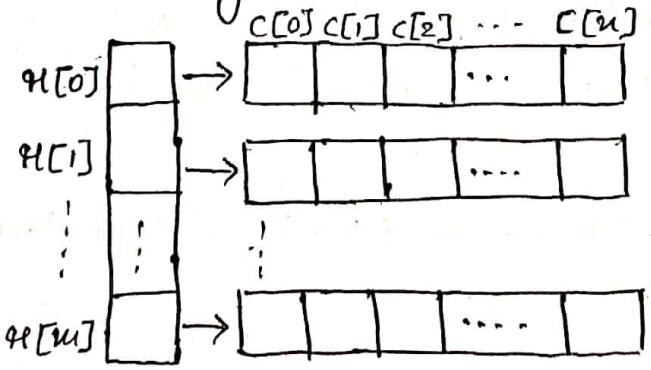
Nithin, VVCE Mysu

Scanned with CamScanner

```
arr = (int*) malloc (n * sizeof (int)); (or) arr = (int*) calloc (n, sizeof (int));
if (arr == NULL)
  { printf ("Memory allocation failed");
  }
else
  { printf ("Enter the Array Elements:");
    for (i=0; i<n; i++)
      { scanf ("%d", &arr[i]);
      }
  }
printf ("The Array Elements are:");
for (i=0; i<n; i++)
  { printf ("%d", arr[i]);
  }
free (arr);
}
```

* 2-D Dynamically Allocated Arrays :- The procedure to be followed to create 2D Dynamic Arrays is not as simple as 1D Dynamic Array Creation.

→ Consider the foll Example, to understand the Creation 2D-Dynamic Array




We create a 1-D Array "arr" whose length is 'n', Each Element of arr is a 1-D array whose length is 'c'.

* Example-program : to read & print 2D-Array dynamically using Malloc() or Calloc().

```
#include <stdio.h>
#include <stdlib.h>
```

Nithin, VVCE Mysuru

Scanned with CamScanner

```
Void main()
{
  int r, c, i, j;
  printf("Enter the rows & cols:");
  scanf("%d %d", &r, &c);
  int *arr[r];
  for(i=0; i<r; i++)
  {
    arr[i] = (int*) malloc(c * sizeof(int));
  }
  for(i=0; i<r; i++)
  { for(j=0; j<c; j++)
    { scanf("%d", &arr[i][j]);
    }
  }
  printf("2-D Array:");
  for(i=0; i<r; i++)
  { for(j=0; j<c; j++)
    { printf("%d", arr[i][j]);
    }
    printf("\n");
  }
  free(arr);
}
```

* Sparse Matrix :- A Matrix can be defined as 2D Array with "n" columns & "n" rows Represents a nxn Matrix. There may be a Situation in which a Matrix Contains More no of zero Values than non-zero Values such Matrix is known as "Sparse Matrix".

→ Example

$$\begin{bmatrix} 10 & 0 & 0 & 40 \\ 11 & 0 & 22 & 0 \\ 0 & 0 & 0 & 0 \\ 20 & 0 & 0 & 50 \\ 0 & 15 & 0 & 25 \end{bmatrix}$$

Nithin, VVCE Mysuru

Scanned with CamScanner

* Triplet Representation of Sparse Matrix → We consider only non-zero values along with their row & column indexes - values.

→ In this representation, the 0th row stores "total no of rows", "total no of columns" & "total no of non-zero values" in Matrix.

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 9 | 0 |
| 1 | 0 | 8 | 0 | 0 | 0 | 0 |
| 2 | 4 | 0 | 0 | 2 | 0 | 0 |
| 3 | 0 | 0 | 0 | 0 | 0 | 5 |
| 4 | 0 | 0 | 2 | 0 | 0 | 0 |

⇒

| Rows | col | Values |
|---|---|---|
| 5 | 6 | 6 |
| 0 | 4 | 9 |
| 1 | 1 | 8 |
| 2 | 0 | 4 |
| 2 | 3 | 2 |
| 3 | 5 | 5 |
| 4 | 2 | 2 |

(Triplet Repr$^n$)

In above Example Matrix, there are only 6 non-zero Elements. The first row of triplet repr$^n$ is filled with values 5, 6 & 6 which indicates that it is a Sparse Matrix with 5-rows, 6-col & 6-nonzero values.
Second row is filled with 0, 4, 9 which indicates the value in the Matrix at 0th row, 4th col is '9' & so on.

* Example - program → to check the given Matrix is Sparse or not.

```
#include <stdio.h>
Void main()
{ int arr[10][10];
  int i, j, m, n, count=0;
  printf("Enter the order of Matrix\n");
  scanf("%d %d", &m, &n);
  printf("Enter the Elements of Matrix");
    for(i=0; i<m; i++)
      { for(j=0; j<n; j++)
```

Nithin, VVCE, Mysuru

Scanned with CamScanner

```
        {
            scanf ("%d", &a[i][j]);
        }
    }
    for (i=0; i<m; i++)
    {
        for (j=0; j<n; j++)
        {
            if (a[i][j] == 0)
                count++;
        }
    }
    if (count > ((m*n)/2))
    printf ("Sparse Matrix");
    else
    printf ("Not Sparse Matrix");
}
```

* Example - program → to Compute the Transpose of given Sparse Matrix.

```
#include<stdio.h>
void main()
{
int a[10][10], b[10][10], i, j, m, n;
printf ("Enter the order of Matrix\n");
scanf ("%d %d", &m, &n);
printf ("Enter the Elements of Matrix");
for (i=0; i<m; i++)
{
    for (j=0; j<n; j++)
    {
        scanf ("%d", &a[i][j]);
    }
}
for (i=0; i<m; i++)
{
    for (j=0; j<n; j++)
    {
        b[i][j] = a[j][i];
    }
}
```

Nithin, VVCE Mysuru

Scanned with CamScanner

```
printf("The Transpose of given Matrix:");
for(i=0; i<m; i++)
{
    for(j=0; j<n; j++)
    {
        printf("%d ",b[i][j]);
    }
    printf("\n");
}
}
```

* polynomials :- polynomial object is a Homogenous ordered list of pairs <Exponent, Co-efficient> where Each co-efficient is unique.

→ Operation on polynomial Includes returning the degree, Extr-acting co-efficient, addition, Multiplication Etc.

→ The different types of polynomial operations are
  * Representation
  * Addition
  * Multiplication

→ A polynomial is an Expression that Contains More than two terms. A term is made up of Co-efficient & Exponent.

Ex :- $p(x) = 4x^3 + 6x^2 + 7x + 9$

* Polynomial Representation → A polynomial can be represented by
  * Array
  * Linked List

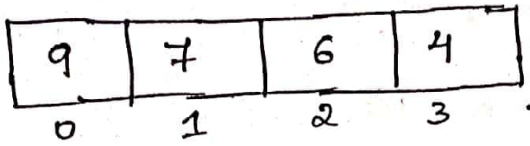
→ "Array representation" assumes that the Exponents of the give-n Expression are arranged from 0 to higher value (degree) which is represented by the Index of array begining with '0'. The Co-efficients of the respective Exponent are placed at the Appropriate Index in the Array.

→ Consider the following Example of polynomial Repres-enting

Nithin, VVCE Mysuru

Scanned with CamScanner

$p(x) = 4x^3 + 6x^2 + 7x + 9$

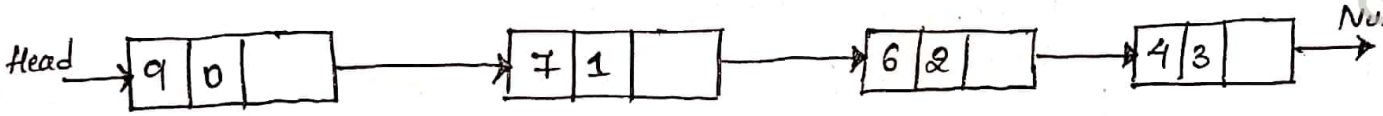



→ A polynomial may also be represented using a linked list, A Structure may be defined such that it Contains two parts One is the Co-Efficient & Second is Corresponding Exponent.

```
Struct polynomial
{ int cf;
  int exp;
  Struct polynomial *next;
};
```

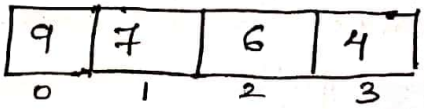
→ Consider the Example of polynomial representation using Linked List:

$p(x) = 4x^3 + 6x^2 + 7x + 9$

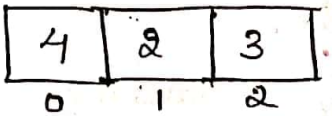



* Addition of two polynomials → Addition of two polynomials using Arrays is Straight-forward Method, Since both the Arrays may be added up Elements wise begining from 0 to n-1, resulting in addition of two polynomials.

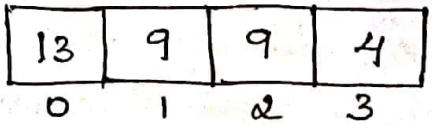
$p(x) = 4x^3 + 6x^2 + 7x + 9$ →




$q(x) = 3x^2 + 2x + 4$ →




$Sum = 4x^3 + 9x^2 + 9x + 13$ →




→ The function to add two polynomials using Array of Structure is as follows:

```
int add_poly (Struct poly p1[10], Struct poly p2[10], Struct
              poly p3[10], int t1, int t2)
{
```

Nithin, VCE Mysuru

Scanned with CamScanner

```
Put i=0, j=0, k=0;
While (i<t1 && j<t2)
{ if (p1[i].Exp == p2[j].Exp)
    {
      p3[k].cf = p1[i].cf + p2[i].cf;
      p3[k].Exp = p1[i].Exp;
      i++; j++; k++;
    }
  Else if (p1[i].Exp > p2[j].Exp)
    {
      p3[k].cf = p1[i].cf;
      p3[k].Exp = p1[i].Exp;
      i++; k++;
    }
  Else
    {
      p3[k].cf = p2[j].cf;
      p3[k].Exp = p2[j].Exp;
      j++; k++;
    }
  }
}
```

* String processing & pattern Matching Algorithms :- Text Editing programs frequently need to find all occurances of a pattern in the text.
The pattern searched for a particular word is supplied by the user.

→ "String Matching" can greatly increase the responsiveness of text-editing programs.
Some good Examples like "DNA Sequence pattern" & "Internet Search Engines" will make use of String Matching Algorithm.

→ The following are two Algorithms discussed in next sections:
* Naive pattern Searching (Brute-Force Algo)
* Knuth Morris pratt (KMP) Algorithm

Nithin, VVCE Mysuru

Scanned with CamScanner

* Naive pattern Matching → The Naive String Matching procedure will be Sliding a template containing the pattern over the text, noting for which Shifts all of the characters on the template Equals the Corresponding character in the text.

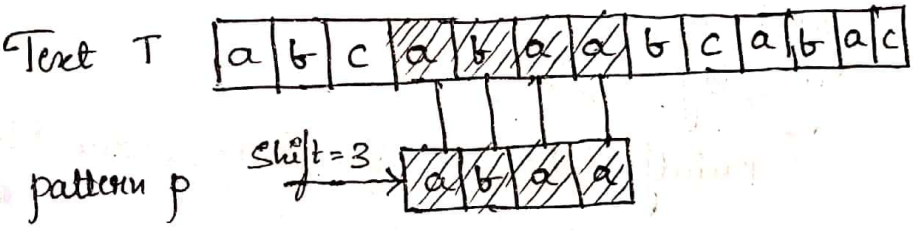
→ String-Matchers Running time Equals its Matching time because it requires no preprocessing.
The Naive Algorithm finds all valid Shifts using a loop that Checks the Condition

$$p[1 \ldots m] == T[s+1 \ldots s+m]$$

* Consider the following Example:

p = | a | b | a | a |

T = | a | b | c | a | b | a | a | b | c | a | b | a | c |




* Algorithm: Naive_String_Match (T, P)

1. n = T.length
2. m = p.length
3. for shift s = 0 to n-m
   {
   if p[1 .... m] == T[s+1 .... s+m]
   printf ("pattern occurs with shift %d", s);
   }

* Example-program: To Search given pattern in the String using Naive pattern Matching (Brute-force Algo).

```
#include <stdio.h>
char str[100], pat[50];
int i, m, flag=0;
void String_match()
```

Nithin, VVCE Mysu

Scanned with CamScanner

```
{
  i = m = 0;
  while (str[m] != '\0')
    { if (str[m] == pat[i])
      { i++; m++;
        if (pat[i] == '\0')
          { flag = 1;
          }
      }
    }
}
void main()
{
  printf("Enter the Main String");
  gets(str);
  printf("Enter the pattern");
  gets(pat);
  if (flag == 1)
    { printf("pattern found in Main String");
    }
  Else
    { printf("pattern Not found");
    }
}
```

* Knuth Morris pratt (KMP) Algorithm :- Uses the "prefix" of the given pattern & improves the performance of String Matching.

The Basic idea behind KMP Algorithm is whenever we detect a Mismatch, we already know some of the characters in the text of Next window. We take advantage of this information to avoid Matching the character that we know will anyway Match.

→ The rules to be followed for prefix code Computation :

* The prefix code of first character is always '0'.

Nithin, VVCE Mysuru

Scanned with CamScanner

* Initialize to Index Variable i & j, such that i=0 & j=1.
* Whenever the character present at Index i & j matches Increment both i & j by 1.
  prefix value of Matched character will be Index value of 'i' + 1.
* Whenever the character present at Index i & j miss-atches,
  Check the prefix value of previously matched at Index 'j', i goto that & try to Match, If matches Increment the prefix by i+1. Else follow the same until 'i' reaches 0.

* Some prefix-code Computation Examples →

| Index | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| pat | A | B | C | A | B |
| prefix | 0 | 0 | 0 | 1 | 2 |

| Index | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| pat | A | B | A | B | A | C |
| prefix | 0 | 0 | 1 | 2 | 3 | 0 |

| index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| pat | B | A | B | A | B | A | C | B | B | A |
| prefix | 0 | 0 | 1 | 2 | 3 | 4 | 0 | 1 | 1 | 0 |

| Index | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| pat | A | B | A | A | B | A |
| prefix | 0 | 0 | 1 | 1 | 2 | 3 |

| index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| pat | A | B | A | B | A | B | A | B | C | A |
| prefix | 0 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 0 | 1 |

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| pat | X | Y | X | Y | X | Z | X |
| prefix | 0 | 0 | 1 | 2 | 3 | 0 | 1 |

| Index | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| pat | a | b | a | b | a | a |
| prefix | 0 | 0 | 1 | 2 | 3 | 1 |

| Index | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| pat | A | B | C | D | A | B | D |
| prefix | 0 | 0 | 0 | 0 | 1 | 2 | 0 |

Nithin, VCE Mysuru

Scanned with CamScanner

* Apply KMP Algorithm for the given

Text : A B A B C A B C A B A B A B D

pattern : A B A B D

⇒ Firstly, we need to compute the prefix table for the given pattern.

| Index | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| pat | A | B | A | B | D |
| prefix | 0 | 0 | 1 | 2 | 0 |

→ Next, Apply the prefix code to perform String Matching

i i i i (i)

*Iteration-1 :- A B A B C A B C A B A B A B D

Mismatch occured in place of 'd'. We've 'c' there, upto prefix value = 2 the Match was Successful so directly Compare the character at Index = 2, if it matches Continue the Matching, Otherwise Repeat the Step until you reach Index '0' Once you reach Index in pattern = 0, Increment 'i' by 1 & Start fresh Match.

i i (i)

*Iteration-2 :- A B A B C A B C A B A B A B D

Again make use of prefix values to reuse the previously Matched character, if its not possible, increment i by 1.

i i i i (i)

*Iteration-3 :- A B A B C A B C A B A B A B D

i i i i i

Mismatch occured in place of 'D', Corresponding prefix value = 2, so directly Start Matching character at Index = 2 of pattern, here its matching so continue. Finally you find pattern.

* Apply KMP - String Matching for the foll:

Text : A B C X A B C D A B X A B C D A B C D A B C Y

pattern : A B C D A B C Y

Nithin, VVCE Mysuru

Scanned with CamScanner

⇒ Firstly, generate the prefix table for given pattern

| index | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| pat | A | B | C | D | A | B | C | Y |
| prefix | 0 | 0 | 0 | 0 | 1 | 2 | 3 | 0 |

* Iteration-1 :- A B C X A B C D A B X A B C D A B C D A B C Y

* Iteration-2 :- A B C X A B C D A B X A B C D A B C D A B C Y

* Iteration-3 :- A B C X A B C D A B X A B C D A B C D A B C Y

Mismatch occurred in place of $Y \neq D$, the prefix value of previous 'C' = 3, so directly Start Matching unmatched character with character at index = 3 in pattern. Finally pattern found.

* Apply KMP Algorithm for the given

Text :- C A B A B A B C A B A B A C A

pattern: A B A B A C

⇒ Firstly, generate the prefix table for given pattern

| index | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| pat | A | B | A | B | A | C |
| prefix | 0 | 0 | 1 | 2 | 3 | 0 |

* Iteration-1 :- C A B A B A B C A B A B A C A

* Iteration-2 :- C A B A B A B C A B A B A C A

* Iteration-3 :- C A B A B A B C A B A B A C A

* Algorithm :- KMP-Matcher (T, P)

1. n = T.length
2. m = P.length

Nithin, VVCE Mysuru

Scanned with CamScanner

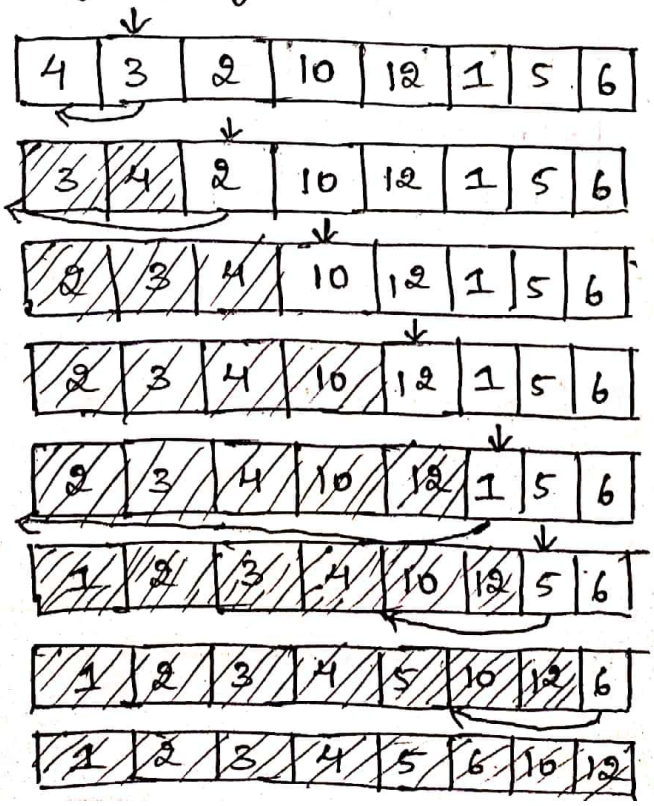
```
3. π = Compute_prefix_Value (p)
4. q = 0
5. for i = 1 to n
       while q > 0 & p[q+1] ≠ T[i]        // Mismatch occurred & prefix > 0, Start
           q = π[q]                          Comparison from
                                             prefix = index[pattern]
6. if p[q+1] == T[i]
7.     q = q+1
8. if q == m
       print "pattern found".
```

Nithin, VVCE Mysuru

* Insertion Sort :- Insertion Sort is a Simple Sorting Algorithm that works Similar to the way you Sort playing Cards in your hands.

→ The Array is Virtually Split into a Sorted & an Unsorted part. Values from the Unsorted part are picked & placed at the Correct position in the Sorted part.

→ Consider the following Example on Insertion Sort




Scanned with CamScanner

→ The working procedure of Insertion Sort :-

* Sorting will start from the element at index = 1 & that element will be considered as key, 'j' value will be 'i-1'

```
for (i=1; i<n; i++)
    key = a[i];
    j = i-1;
```

* when 'j' value is greater or Equal to zero & element at 'j' index is greater than key, Swap a[j] with a[j+1] & decrement 'j' by 1

```
while (j>=0 && a[j] > key)
    a[j+1] = a[j];
    j = j-1
```

the above logic will restrict the program to Compare key value with only left part of array by decrementing 'j'.

* Once the conditions, Either 'j' reaches '-1' or key is greater than Element at index 'j' the loop fails & copies the Content of key to "j+1" index.

```
a[j+1] = key;
```

* program :- to Sort the given list of numbers using Insertion Sort.

```
#include <stdio.h>
Void main ()
{
int i, j, key, a[50], n;
printf ("Enter the number of Array Elements");
scanf ("%d", &n);
printf ("Enter the Array Elements: ");
for (i=0; i<n; i++)
  {
  scanf ("%d", &a[i]);
  }
```

Nithin, VVCE Mysuru

Scanned with CamScanner

```
for (i=1; i<n; i++)
  {
    key = a[i];
    j = i-1;
    while (j>=0 && a[j] > key)
      {
        a[j+1] = a[j];
        j = j-1;
      }
    a[j+1] = key;
  }
printf ("Sorted Array Elements: ");
for (i=0; i<n; i++)
  {
    printf ("%d", a[i]);
  }
}
```

* Radix Sort :- The idea of Radix Sort is to do digit by digit Sort Starting from Least Significant digit to most Significant digit.

Radix Sort uses Counting Sort / Bucket Sort as a Subroutine to Sort.

→ Consider the following Example on Radix-Sort

170, 45, 75, 90, 802, 24, 2, 66

* Iteration-1 (1's place) output

170, 90, 802, 2, 24, 45, 75, 66

* Iteration-2 (10's place) output

802, 02, 24, 45, 66, 170, 75, 90

* Iteration-3 (100's place) output

002, 024, 045, 066, 075, 090, 170, 802

Sorted List ⇒ 2, 24, 45, 66, 75, 90, 170, 802 //

Nithin, VVCE Mysuru

Scanned with CamScanner

→ The Working procedure of Radix-Sort :- Consider the foll Small list of no's to demonstrate the working of Radix Sort.

arr = | 768 | 1 | 34 | 9 |

*Step-1 → find the Maximum Value present in the given list

```
int getmax (int arr[], int n)
{ int i, m = arr[0];
  for (i=1; i<n; i++)
  { if (arr[i] > m)
    { m = arr[i];
    }
  }
return m;
}
```

In the above Example maximum value m=768.

*Step-2 → the Radix Sort fixes the no of Iterations bas-ed on the Value of 'm' & 'Exp' will start with '1' & gets Multiplied by 10 in Each Iteration.

for (Exp = 1; m/Exp > 0; Exp *= 10)

the above loop Executes for 3 times as m = 768

$768/1 > 0$, $768/10 > 10$, $768/100$, $768/1000$ ✗ (Stops)

→ In Every Iteration two additional Array is Created i.e "output" & "count", where Count array will be filled with Zero's.

```
int output[4];
int count[10] = {0};
```

the "output" Array Size should be Same as Input, the "Count" Array Size should be 10.

output = | 0 | 1 | 2 | 3 |

Count = | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Nithin, VVCE Mysuru

Scanned with CamScanner

→ Update the Count of Count Array by Applying following operation

```
for (i=0; i<n; i++)
{
    Count [(arr[i]/Exp) % 10]++;
}
```

Count =

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |

→ Compute the Cumulative Count on Count Array

```
for (i=1; i<10; i++)
{
    Count [i] = Count[i] + Count [i-1];
}
```

Count =

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 2 | 2 | 2 | 2 | 3 | 4 |

→ Assign the output Array Value by the End (n-1) of Input Array

```
for (i=n-1; i>=1; i--)
{
    output [count [(arr[i]/Exp) % 10] - 1] = arr[i];
    Count [(arr[i]/Exp) % 10]--;
}
```

output =

| 0 | 1 | 2 | 3 |
|---|---|---|---|
| 1 | 34 | 768 | 9 |

→ Copy the Contents of output Array to Input Array

```
for (i=0; i<n; i++)
{
    arr[i] = output[i];
}
```

arr =

| 0 | 1 | 2 | 3 |
|---|---|---|---|
| 1 | 34 | 768 | 9 |

→ Apply the "Step-2" by Multiplying "Exp" Value by 10, i.e Exp=10 and Exp=100, finally the list will be Sorted list.

* programs:- to Sort the given list of numbers using Radix Sort.

```
#Include <stdio.h>
```

Nithin, VVCE, Mys

Scanned with CamScanner

```
int getmax (int arr[], int n)
{
    int mx = arr[0], i;
    for (i=1; i<n; i++)
    {
        if (arr[i] > mx)
        {
            mx = arr[i];
        }
    }
    return mx;
}
int main()
{
    int arr[10], i, n;
    printf ("Enter the Size of array");
    scanf ("%d", &n);
    printf ("Enter the Array Elements:");
    for (i=0; i<n; i++)
    {
        scanf ("%d", &arr[i]);
    }
    int m = getmax(arr, n);
    int Exp;
    for (Exp=1; m/Exp>0; Exp*=10)
    {
        int output[10];
        int count[10] = {0};
        for (i=0; i<n; i++)
        {
            count [(arr[i]/Exp)%10]++;
        }
        for (i=1; i<10; i++)
        {
            count[i] = count[i] + count[i-1];
        }
        for (i=n-1; i>=0; i--)
        {
            output [count [(arr[i]/Exp)%10]-1] = arr[i];
            count [(arr[i]/Exp)%10]--;
        }
        for (i=0; i<n; i++)
        {
            arr[i] = output[i];
        }
    }
    printf ("Sorted Array Elements:");
    for (i=0; i<n; i++)
    {
        printf ("%d", arr[i]);
    }
    return 0;
}
```

Nithin, VVCE Mysuru

Scanned with CamScanner

* String operations :- Strings are defined as an array of characters terminated with a special character '\0'.

→ Some of the most commonly used string operations are

* String length (strlen)
* String Concatenate (strcat)
* String Compare (strcmp)
* String reverse (strrev)
* String Substring (strstr)

* program :- on String length

```
#include <stdio.h>
void main()
{
  char str[100];
  int i, length = 0;
  printf("Enter a String");
  gets(str);
  for(i=0; str[i]!='\0'; i++)
    length++;
  printf("length = %d", length);
}
```

* program :- on String Concatenate

```
#include <stdio.h>
void main()
{
  char str1[100], str2[100], i, j;
  printf("Enter first String");
  gets(str1);
  printf("Enter second String");
  gets(str2);
  for(i=0; str1[i]!='\0'; i++)
    for(j=0; str2[j]!='\0'; j++)
      {
        str1[i] = str2[j];
      }
```

Nithin, VVCE Mysuru

Scanned with CamScanner

```
        str1[i] = '\0';
    }
    printf("Concatenated String = %s", str1);
}
```

* program: on String Compare

```
#include<stdio.h>
void main()
{
    char str1[100], str2[100];
    int i;
    printf("Enter two Strings");
    gets(str1);
    gets(str2);
    i=0;
    while(str1[i]==str2[i] && str1[i]!='\0')
        i++;
    if(str1[i] > str2[i])
        printf("str1 is greater");
    else if(str1[i] < str2[i])
        printf("str2 is greater");
    else
        printf("Both are Same");
}
```

* program: to reverse given String

```
#include<stdio.h>
void main()
{
    char s[100], r[100];
    int begin, end, count=0;
    printf("Enter a String");
    gets(s);
    while(s[count]!='\0')
    {
        count++;
    }
    end = count-1;
    for(begin=0; begin<count; begin++, end--)
    {
        r[begin] = s[end];
    }
```

Nithin, VVCE Mysuru

Scanned with CamScanner

```
        r[begin] = '\0';
        printf("reverse = %s", r);
    }
```

* program : to find the Substring

```
#include <stdio.h>
void main()
{
    char str[100], sub[10];
    int pos, len, c=0;
    printf("Enter a String");
    gets(str);
    printf("Enter the position & length of Substring");
    scanf("%d %d", &pos, &len);
    while (c < length)
    {
        sub[c] = str[pos+c-1];
        c++;
    }
    sub[c] = '\0';
    printf("Required Substring is %s", sub);
}
```

* Note → Along with these programs, prepare for program-1 from Lab cycle.

Nithin, VVCE Mysuru

Scanned with CamScanner